

Python

Summary Session 12



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Для чего нужна документация?



Документация

Это описание кода, которое помогает разработчикам понимать, как работают функции и другие компоненты программы. Она делает код более понятным, облегчает его поддержку и расширение



Docstrings (Документирующие строки)

Это строки документации, заключённые в тройные кавычки (""" или '''), которые используются для описания функций и других компонентов

Docstrings (Документирующие строки)



В отличие от комментариев, `docstrings` являются частью функции и могут быть получены через `help()`

Docstrings



Синтаксис

```
def function_name(param1, param2):
    """
    Описание функции.

    :param param1: Описание первого
    параметра.
    :param param2: Описание второго
    параметра.
    :return: Описание возвращаемого
    значения.
    """
    pass
```

Пример

```
def greet(name):
    """
    Функция принимает имя и возвращает
    строку приветствия.

    :param name: Имя пользователя.
    :return: Приветственное сообщение.
    """
    return f"Hello, {name}!"
```

Экспресс-опрос

?

Какая функция используется для просмотра встроенной и пользовательской документации объектов?



Функция help

Эта функция используется для просмотра встроенной и пользовательской документации объектов, например функций. Она выводит `docstrings` объекта, если они определены.

Docstrings



Синтаксис

`help(object)`

Пояснения

- `object` — любой объект Python, для которого нужно получить справку.
- Если объект не указан, откроется интерактивный справочный режим

Пример: Получение справки по встроенной функции



```
help(sum)
```

Help on built-in function sum in module builtins:

```
sum(iterable, /, start=0)
```

Return the sum of a 'start' value (default: 0) plus an iterable of numbers

When the iterable is empty, return the start value.

This function is intended specifically for use with numeric values and may reject non-numeric types.

Пример: Просмотр документации пользовательской функции



```
help(greet)
```

```
Help on function greet in module __main__:
```

```
greet(name)
```

```
    Функция принимает имя и возвращает строку приветствия.
```

```
    :param name: Имя пользователя.
```

```
    :return: Приветственное сообщение.
```

Пример: Получение списка методов объекта



```
# Выведет полное описание всех методов строк в Python  
help(str)
```

Функция `help()` также показывает методы и атрибуты объектов.

Экспресс-опрос

?

Что произойдет, если вызвать `help()` без аргумента?

Вызов справки без аргументов



Если вызвать `help()` без аргумента, откроется интерактивный режим справки

После этого можно вводить имена объектов для получения информации



ВОПРОСЫ



Экспресс-опрос

?

Для чего нужны аннотации типов?



Аннотации типов

Это механизм, позволяющий указывать ожидаемые типы аргументов и возвращаемого значения функции, а также типы переменных. Они помогают сделать код понятнее, повысить читаемость и упростить отладку, но не накладывают жёстких ограничений на типы данных

Динамически типизированные языки



Python остаётся **динамически типизированным языком**, и аннотации типов **не заставляют интерпретатор проверять соответствие типов** во время выполнения

Зачем нужны аннотации типов?



Упрощение понимания кода — легче понять, какие данные ожидаются на входе и выходе



Помощь в отладке — статический анализ (проверка кода без выполнения) может обнаружить потенциальные ошибки



Автодокументирование — IDE могут использовать аннотации для подсказок

Аннотации типов



Синтаксис

```
def function_name(param1: type1, param2:
type2, ...) -> return_type:
    # тело функции
    return value

variable: type = value
```

Пояснения

- param1, param2 — имена параметров функции
- type1, type2, type — ожидаемые типы данных для соответствующих параметров или переменных
- -> return_type — аннотация типа возвращаемого значения

Пример аннотации типов



```
def add(a: int, b: int) -> int:  
    return a + b
```

```
num: int = 10
```

- `a: int` — аргумент `a` должен быть целым числом
- `b: int` — аргумент `b` должен быть целым числом
- `-> int` — функция должна возвращать целое число
- `num: int` — в переменной можно хранить только целое число

Экспресс-опрос

?

Влияют ли аннотации на выполнение кода?

Аннотации не влияют на выполнение кода



```
def add(a: int, b: int) -> int:
    return a + b

print(add(3, 5))
# не приведёт к ошибке, хотя переданы строки
print(add("3", "5"))
```



ВОПРОСЫ



Аннотации для базовых типов



В большинстве случаев название аннотации совпадает с названием типа данных, который она обозначает. Это делает код интуитивно понятным и легко читаемым

Аннотация целых чисел (int)



```
def factorial(n: int) -> int:
    """Возвращает факториал числа."""
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Аннотация чисел с плавающей точкой (float)



```
def convert_to_celsius(fahrenheit: float) -> float:
    """Конвертирует температуру из градусов Фаренгейта в Цельсий."""
    return (fahrenheit - 32) * 5 / 9
```

Аннотация строк (str)



```
def greet(name: str) -> str:  
    """Возвращает приветственное сообщение."""  
    return f"Hello, {name}!"
```

Аннотация логических значений (bool)



```
def is_even(number: int) -> bool:  
    """Определяет, является ли число чётным."""  
    return number % 2 == 0
```


Аннотация None (функция без возврата)



```
def log_message(message: str) -> None:
    """Выводит сообщение в консоль, но не возвращает значения."""
    print(f"LOG: {message}")
```

Экспресс-опрос

?

Можно ли в Python 3.9+ аннотировать структуры данных?

Аннотации для структур данных



В Python 3.9+ можно аннотировать списки, кортежи, множества и словари просто указывая встроенные типы. Однако важно понимать, как именно указывать тип содержимого и какие особенности у разных структур

Списки (list)



Синтаксис

`list[element_type]`

Пояснения

- В скобках указывается тип элементов внутри списка
- Рекомендуется указывать тип содержимого

Пример аннотации для списка (list)



```
def process_numbers(numbers: list[int]) -> list[int]:  
    # Список чисел  
    return [n ** 2 for n in numbers]
```

Кортежи (tuple)

Синтаксис



```
tuple[type1, type2]
```

Пояснения

Если кортеж содержит фиксированное число элементов

```
tuple[element_type, ...]
```

Если длина кортежа не фиксирована

Пример аннотации для кортежа (tuple)



```
def get_info() -> tuple[str, float]:
    # Первый элемент str, второй элемент float
    return "Bob", 4.91

def variable_tuple() -> tuple[int, ...]:
    # Кортеж произвольной длины, но только целые числа
    return 5, 8, 2
```

Множества (set)



Синтаксис

`set[element_type]`

Пояснения

- В скобках указывается тип всех элементов множества
- Рекомендуется указывать тип содержимого

Пример аннотации для множества (set)



```
def unique_chars(text: str) -> set[str]:  
    # Множество уникальных символов  
    return set(text)
```

Замороженные множества (frozenset)



Синтаксис

`frozenset[element_type]`

Пояснения

- Работает аналогично `set`, но создаёт неизменяемое множество
- Рекомендуется указывать тип содержимого

Пример аннотации для замороженного множества (frozenset)



```
def frozen_example() -> frozenset[int]:  
    # Множество уникальных чисел  
    return frozenset([1, 2, 3])
```

Словари (dict)



Синтаксис

```
dict[key_type, value_type]
```

Пояснения

- В скобках указываются типы ключей и значений
- Рекомендуется указывать типы содержимого

Пример аннотации для словаря (dict)



```
def count_words(text: str) -> dict[str, int]:
    """Принимает строку и возвращает словарь с подсчётом каждого слова."""
    words = text.split()
    # Словарь, где ключи – строки, значения – целые числа
    return {word: words.count(word) for word in words}
```

Коллекции без указания типов



Если тип элементов в коллекции не важен или вы хотите создать коллекцию с элементами разных типов, типы можно не указывать

Пример для коллекции без указания ТИПОВ



```
# Тип элементов не указан
def process_data(data: list) -> list:
    return [d for d in data if d]

# Элементы должны быть числами
def process_numbers(numbers: list[int]) -> list[int]:
    return [n ** 2 for n in numbers]
```

Экспресс-опрос

?

Чем отличаются аннотации в версиях до Python 3.9?

Важно



В версиях Python до 3.9 для аннотации структур данных использовались **специальные классы из модуля typing**:

- List вместо list
- Tuple вместо tuple
- Set вместо set
- FrozenSet вместо frozenset
- Dict вместо dict

Аннотация в старых версиях



В Python 3.9+ (рекомендуется)

```
def process_numbers(numbers: list[int]) -> list[int]:
    return [n ** 2 for n in numbers]
```

В Python <3.9 (старый стиль)

```
from typing import List
```

```
def process_numbers_old(numbers: List[int]) -> List[int]:
    return [n ** 2 for n in numbers]
```



ВОПРОСЫ



Аннотации Any, Union и Optional



При аннотировании функций и переменных в Python бывают случаи, когда необходимо указывать **разные возможные типы**. Для этого используются Any, Union и Optional из модуля typing

Экспресс-опрос

?

Что означает аннотация Any?



Any

Эта аннотация означает, что переменная или аргумент могут быть любого типа

Пример для Any



```
from typing import Any

def process_data(data: Any) -> str:
    """Принимает данные любого типа и возвращает строку с их представлением."""
    return f"Данные: {data}"

print(process_data(42))
print(process_data("Hello"))
print(process_data([1, 2, 3]))
```

Экспресс-опрос

?

Что означает аннотация Union?



Union

Эта аннотация позволяет указать, что переменная может содержать один из нескольких типов

Пример для Union



```
from typing import Union

def calculate(value: Union[int, float]) -> float:
    """Принимает число (целое или дробное) и возвращает его квадрат."""
    return value ** 2
print(calculate(5))
print(calculate(2.5))
```



Optional

Аннотация `Optional[T]` — сокращённая запись для `Union[T, None]`, означающая, что значение может быть `None`

Пример для Optional



```
from typing import Optional

def get_user_name(user_id: int) -> Optional[str]:
    """Возвращает имя пользователя или None, если пользователь не найден."""
    users = {1: "Alice", 2: "Bob"}
    return users.get(user_id) # Может вернуть None

print(get_user_name(1))
```

Экспресс-опрос

?

В каких случаях стоит использовать аннотацию Optional?

Когда использовать Optional?



Если функция может не возвращать значение



Если переменная может содержать объект или None



Когда значение может отсутствовать (например, поиск в базе данных)



Union с оператором |

В Python 3.10 появился более краткий синтаксис для аннотаций типов — вместо Union можно использовать оператор |

Пример Union с оператором |



```
def calculate(value: int | float) -> float:
    """Принимает число (целое или дробное) и возвращает его квадрат."""
    return value ** 2

print(calculate(5))
print(calculate(2.5))
```


Экспресс-опрос

?

Для чего используется аннотация Callable?



Callable

Эта аннотация используется для аннотирования аргументов или возвращаемого значения, если параметр должен быть функцией

Пример аннотации callable



```
from typing import Callable

def square(x: float) -> float:
    return x * x

def cube(x: float) -> float:
    return x * x * x

def apply_function(func: Callable, value: float) -> float:
    return func(value)

print(apply_function(square, 5))
print(apply_function(cube, 5))
```

Экспресс-опрос

?

На что влияет изменяемость аргументов функции?

Изменяемые и неизменяемые аргументы функции



Изменяемость передаваемых в функцию объектов влияет на то, будут ли внесённые в аргументы внутри функции изменения затрагивать оригинальные объекты

Передача по значению (копия)

Это передача ссылки на объект, который нельзя изменить. Если объект изменяется внутри функции, создаётся новый объект, а оригинал остаётся неизменным

Применяется к неизменяемым объектам (int, float, bool, str, tuple, frozenset)

Пример передачи по значению (копия, неизменяемый объект)



```
def modify_value(n: int) -> None:
    """Изменяет значение переменной внутри функции (не влияет на оригинал)."""
    print(f"До изменения в функции: {n}, id: {id(n)}")
    n += 1 # Создаётся новый объект
    print(f"После изменения в функции: {n}, id: {id(n)}")

num = 10
modify_value(num)
print(f"Вне функции: {num}, id: {id(num)}")
```

Передача по ссылке (оригинал изменяется)

Это передача ссылки на объект, который можно изменить. Изменение внутри функции затрагивает сам объект

Применяется к изменяемым объектам (list, dict, set)

Пример передачи по ссылке (оригинал изменяется, изменяемый объект)



```
def modify_list(lst: list) -> None:
    """Добавляет элемент в список (изменяет оригинальный объект)."""
    print(f"До изменения в функции: {lst}, id: {id(lst)}")
    lst.append(99) # Изменение оригинального списка
    print(f"После изменения в функции: {lst}, id: {id(lst)}")

my_list = [1, 2, 3]
modify_list(my_list)
print(f"Вне функции: {my_list}, id: {id(my_list)}")
```

Экспресс-опрос

?

Как избежать нежелательных изменений?

Как избежать нежелательных изменений?



Создание копии `сору()` для поверхностного копирования



Использование `деерсору()` для полного копирования вложенных структур

Пример с copy()



```
def safe_modify_list(lst: list) -> list:
    """Работает с копией списка, оставляя оригинальный неизменным."""
    copy_lst = lst.copy()
    copy_lst.append(99)
    return copy_lst

original_list = [1, 2, 3]
new_list = safe_modify_list(original_list)
print(f"Оригинал: {original_list}")
print(f"Копия: {new_list}")
```



ВОПРОСЫ



Экспресс-опрос

?

Какая структура данных, которая используется для управления порядком выполнения функций в программе?



Стек вызовов (call stack)

Это структура данных, которая используется для управления порядком выполнения функций в программе. Когда функция вызывается, её данные помещаются в стек, а после завершения работы – удаляются из него

Зачем нужны аннотации типов?



LIFO (Last In, First Out) – последняя вызванная функция выполняется первой



Каждый вызов функции добавляет **новый фрейм** в стек вызовов



Когда функция завершается (return), её фрейм удаляется из стека

Пример вложенных вызовов функций

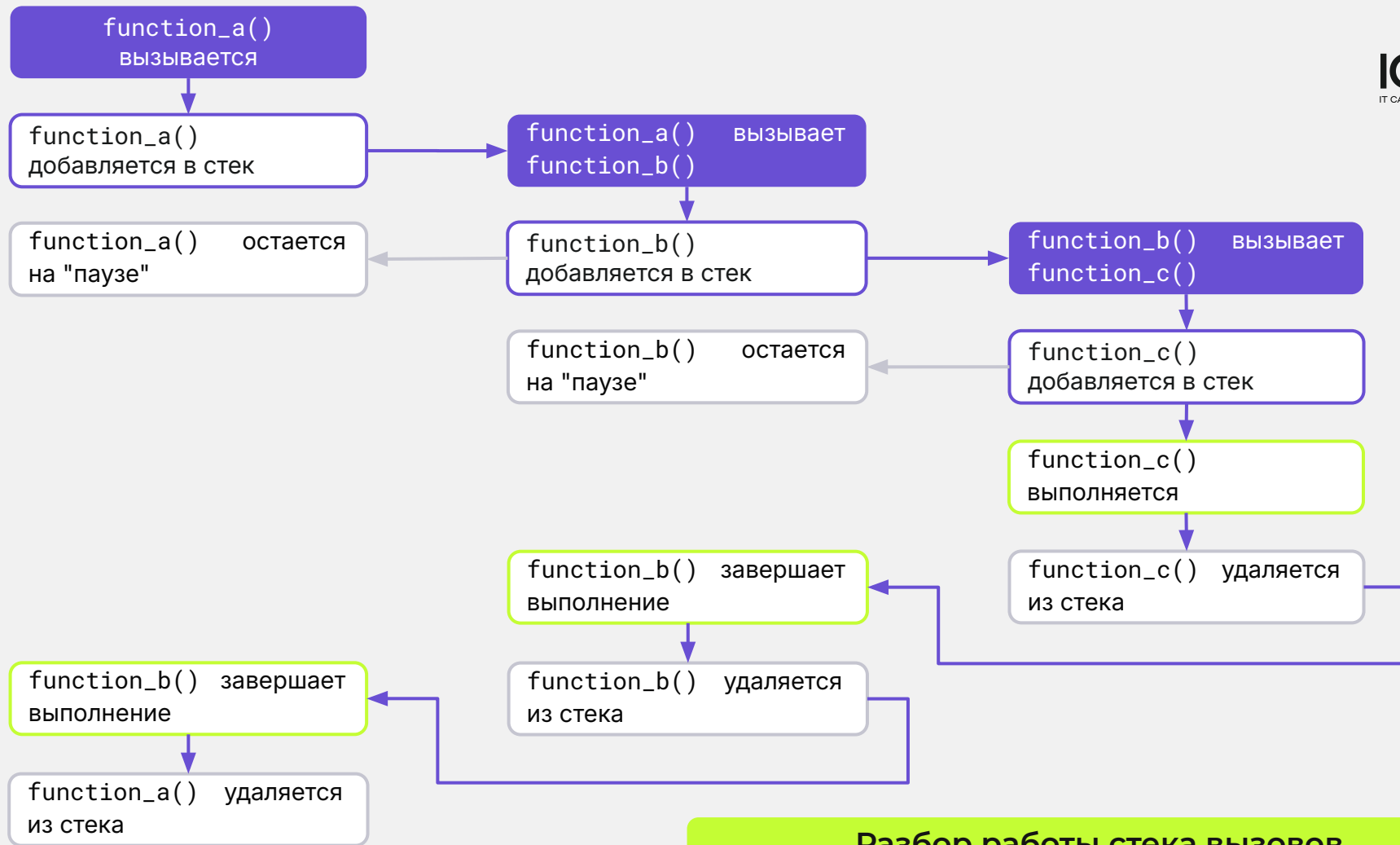


```
def function_a():
    print("Начало A")
    function_b()
    print("Конец A")
```

```
def function_b():
    print("Начало B")
    function_c()
    print("Конец B")
```

```
def function_c():
    print("Начало C")
    print("Конец C")
```

```
function_a()
```





ВОПРОСЫ



Экспресс-опрос

?

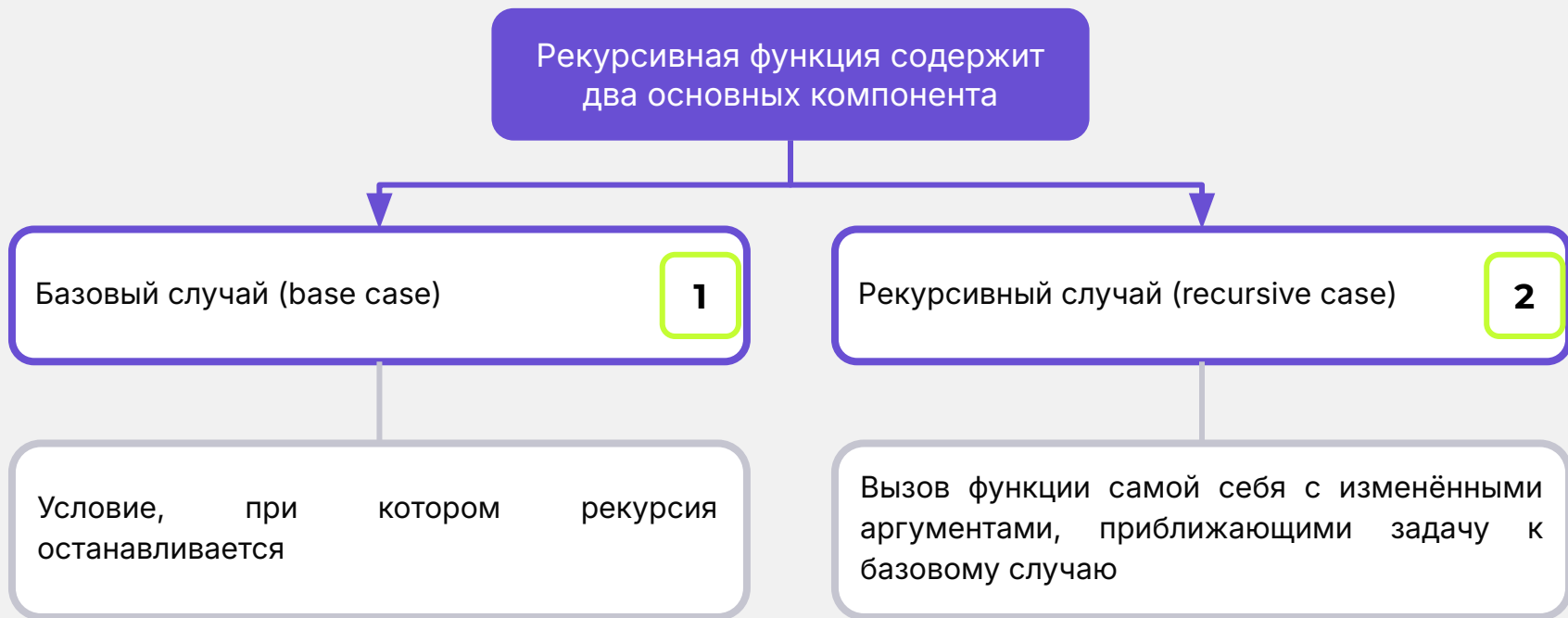
Что такое рекурсия?



Рекурсия

Это подход в программировании, при котором функция вызывает саму себя. Такой подход позволяет разбить сложную задачу на меньшие подзадачи, которые решаются тем же алгоритмом

Принцип работы рекурсивной функции



Пример 1: Простейшая рекурсивная функция (вывод чисел от n до 1)



```
def countdown(n: int) -> None:
    if n <= 0: # Базовый случай
        print("Конец!")
        return
    print(n)
    countdown(n - 1) # Рекурсивный случай
```

```
countdown(5)
```

Часто тот же процесс можно реализовать с помощью **цикла**:

```
def countdown_iterative(n: int) -> None:
    while n > 0:
        print(n)
        n -= 1
    print("Конец!")
countdown_iterative(5)
```

Экспресс-опрос

?

В каком случае возникает `RecursionError`?



RecursionError

Когда функция вызывает саму себя (рекурсия), каждый новый вызов создаёт новый фрейм в стеке, что может привести к переполнению стека

Пример бесконечной рекурсии (не делать так!)



Код

```
def infinite_recursion():  
    infinite_recursion()  
  
infinite_recursion()
```

Пояснения

Этот код вызовет ошибку `RecursionError: maximum recursion depth exceeded`, так как стек вызовов заполнится и программа завершится сбоем

Пример ошибки



RecursionError Traceback (most recent call last)

Cell In[66], line 5

```
1 def infinite_recursion():  
2     infinite_recursion()  
----> 5 infinite_recursion()
```

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

[... skipping similar frames: infinite_recursion at line 2 (2971 times)]

Cell In[66], line 2, in infinite_recursion()

```
1 def infinite_recursion():  
----> 2     infinite_recursion()
```

RecursionError: maximum recursion depth exceeded

Экспресс-опрос

?

Всегда ли в Python аннотации типов заставляют интерпретатор проверять соответствие типов во время выполнения?

Динамически типизированные языки



Python остаётся **динамически типизированным языком**, и аннотации типов **не заставляют интерпретатор проверять соответствие типов** во время выполнения

Пример 1: Простейшая рекурсивная функция (вывод чисел от n до 1)



```
def countdown(n: int) -> None:
    if n <= 0: # Базовый случай
        print("Конец!")
        return
    print(n)
    countdown(n - 1) # Рекурсивный случай
```

```
countdown(5)
```

Часто тот же процесс можно реализовать с помощью **цикла**:

```
def countdown_iterative(n: int) -> None:
    while n > 0:
        print(n)
        n -= 1
    print("Конец!")
countdown_iterative(5)
```

Docstrings



Синтаксис

```
def function_name(param1: type1, param2:
type2, ...) -> return_type:
    # тело функции
    return value

variable: type = value
```

Пояснения

- param1, param2 — имена параметров функции
- type1, type2, type — ожидаемые типы данных для соответствующих параметров или переменных
- -> return_type — аннотация типа возвращаемого значения

Пример рекурсивной функции: Факториал числа



```
def factorial(n: int) -> int:
    if n == 0 or n == 1: # Базовый случай
        return 1
    return n * factorial(n - 1) # Рекурсивный случай

print(factorial(5))
```

Факториал числа $n!$ – это произведение всех чисел от 1 до n .

Экспресс-опрос

?

Что такое бинарный поиск?



Бинарный поиск

Это эффективный алгоритм поиска
элемента в отсортированном списке

Пример рекурсивной функции: Бинарный поиск



```
def binary_search(arr: list[int], target: int, left: int, right: int) -> int:
    if left > right: # Базовый случай: элемент не найден
        return -1
    mid = (left + right) // 2
    if arr[mid] == target:
        return mid
    elif arr[mid] < target:
        return binary_search(arr, target, mid + 1, right) # Поиск в правой части
    else:
        return binary_search(arr, target, left, mid - 1) # Поиск в левой части

array = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(array, 5, 0, len(array) - 1))
print(binary_search(array, 13, 0, len(array) - 1))
print(binary_search(array, 8, 0, len(array) - 1))
```

Экспресс-опрос

?

В чем особенность хвостовой рекурсии?



Хвостовая рекурсия

Это особый вид рекурсии, в котором рекурсивный вызов является последней операцией функции перед возвратом результата

Разница между обычной и хвостовой рекурсией



Обычная рекурсия

```
def factorial(n: int) -> int:
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(5))
```

Хвостовая рекурсия

```
def factorial_tail(n: int, accumulator:
int = 1) -> int:
    if n == 0 or n == 1:
        return accumulator
    return factorial_tail(n - 1, n *
accumulator)

print(factorial_tail(5))
```

Динамически типизированные языки



В Python нет встроенной оптимизации хвостовой рекурсии, которая **заменяет хвостовой рекурсивный вызов на повторное использование текущего кадра стека**. Поэтому рекомендуется **заменять хвостовую рекурсию на итерацию**, чтобы избежать переполнения стека при большом количестве рекурсивных вызовов.

Итеративный вариант факториала



```
def factorial_iterative(n: int) -> int:
    accumulator = 1
    while n > 1:
        accumulator *= n
        n -= 1
    return accumulator

print(factorial_iterative(5))
```




ВОПРОСЫ



Рекурсия и итерация



Это два подхода к решению задач, которые могут быть взаимозаменяемыми в некоторых случаях. Однако выбор между ними зависит от производительности, читабельности кода и ограничений памяти

Экспресс-опрос

?

Какие преимущества рекурсии вы помните? А недостатки?

Преимущества и недостатки рекурсии

Плюсы

- Позволяет разделить сложную задачу на подзадачи
- Упрощает чтение кода (например, обход деревьев и графов)
- Некоторые алгоритмы легче выразить рекурсивно (например, поиск пути в лабиринте)

Минусы

- Может привести к переполнению стека (`RecursionError`)
- Медленнее итеративных решений из-за затрат на вызовы функций
- Использует дополнительную память (каждый вызов создаёт новый стековый фрейм)

Когда использовать рекурсию?



Когда задача делится на подзадачи (разбиение на более мелкие части)



Для работы с деревьями, графами и вложенными структурами



Когда важна выразительность кода (код читается легче, чем итеративная версия)

Экспресс-опрос

?

В каких случаях лучше использовать итерацию?

Когда использовать итерацию?



Когда рекурсия создаёт избыточную нагрузку на стек вызовов



Когда можно решить задачу без хранения промежуточных вызовов



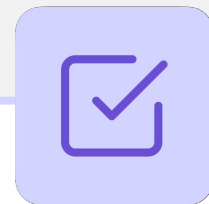
Когда важна эффективность по памяти и скорости



ВОПРОСЫ



Важно



Некоторые задачи сложно или невозможно эффективно решить без использования рекурсии. Это связано с необходимостью **разбиения задачи на более мелкие подзадачи**, а также с работой со сложными вложенными структурами

Экспресс-опрос

?

Что такое дерево?



Дерево

Это иерархическая структура данных, где каждый узел может иметь несколько дочерних узлов

1. Обход дерева (DFS - поиск в глубину)

Задачи, решаемые рекурсией



Рекурсивный подход позволяет обойти все узлы, последовательно углубляясь в каждый из них

Экспресс-опрос



Что такое граф?



Граф

Это структура, состоящая из узлов и рёбер, соединяющих их

2. Обход графа (DFS, рекурсивный поиск путей)

Задачи, решаемые рекурсией



Использование рекурсии в глубинном поиске позволяет эффективно находить пути между вершинами

3. Разбор выражений и синтаксический анализ

Задачи, решаемые рекурсией



При обработке вложенных математических выражений или языковых конструкций удобно использовать рекурсию, так как каждое подвыражение может рассматриваться как самостоятельная подзадача.

4. Ханойские башни

Задачи, решаемые рекурсией



Задача, требующая последовательного перемещения дисков между тремя стержнями с сохранением их порядка. Разбивается на две меньшие аналогичные задачи, что делает рекурсию естественным решением.

5. Разбиение на подмножества и комбинаторика

Задачи, решаемые рекурсией



Задачи, связанные с генерацией всех возможных комбинаций, перестановок и подмножеств множества, решаются с разбиением на подзадачи: выбор одного элемента и рекурсивное построение оставшейся структуры.



ЗАДАНИЕ





Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
help()
```

- a. Выведется документация по всем встроенным функциям
- b. Запустится интерактивный режим справки
- c. None
- d. Ошибка, не передана функция

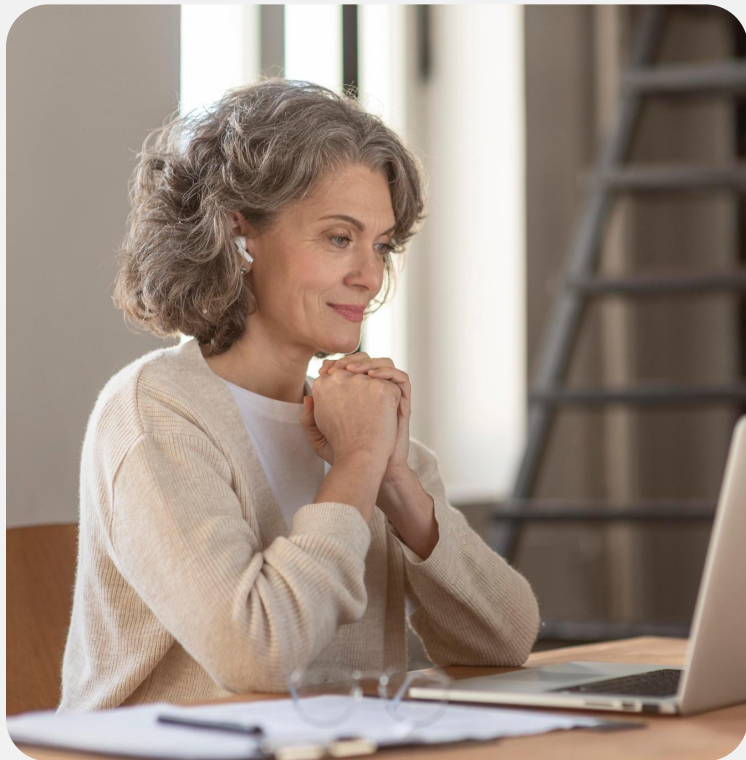


Выберите верный вариант ответа

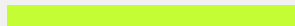
Какой результат будет выведен при выполнении следующего кода?

```
help()
```

- a. Выведется документация по всем встроенным функциям
- b. Запустится интерактивный режим справки**
- c. None
- d. Ошибка, не передана функция



Ответьте на вопрос



Чем docstrings отличаются от обычных комментариев?



Ответьте на вопрос

Чем docstrings отличаются от обычных комментариев?

Ответ:

- docstrings заключены в тройные кавычки (`"""` или `'''`) и являются частью функции.
- Они доступны через `help()`, в отличие от комментариев (`#`), которые игнорируются интерпретатором.



Ответьте на вопрос

Как автоматически сгенерировать шаблон docstrings в PyCharm?



Ответьте на вопрос

Как автоматически сгенерировать шаблон docstrings в PyCharm?

Ответ: Ввести `"""` сразу после `def function():` и нажать Enter.



Найдите ошибку в коде и исправьте её

```
def process_data(data: list[int]) -> list[int]:
    return [d ** 2 for d in data]

print(process_data(5, 7, 8))
```



Найдите ошибку в коде и исправьте её

```
def process_data(data: list[int]) -> list[int]:
    return [d ** 2 for d in data]

print(process_data(5, 7, 8))
```

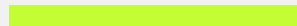
Исправленный вариант:

```
def process_data(data: list[int]) -> list[int]:
    return [d ** 2 for d in data]

print(process_data([5])) # Передаём список, а не
                           несколько чисел
```



Ответьте на вопрос



Можно ли аннотировать аргумент, принимающий несколько типов?

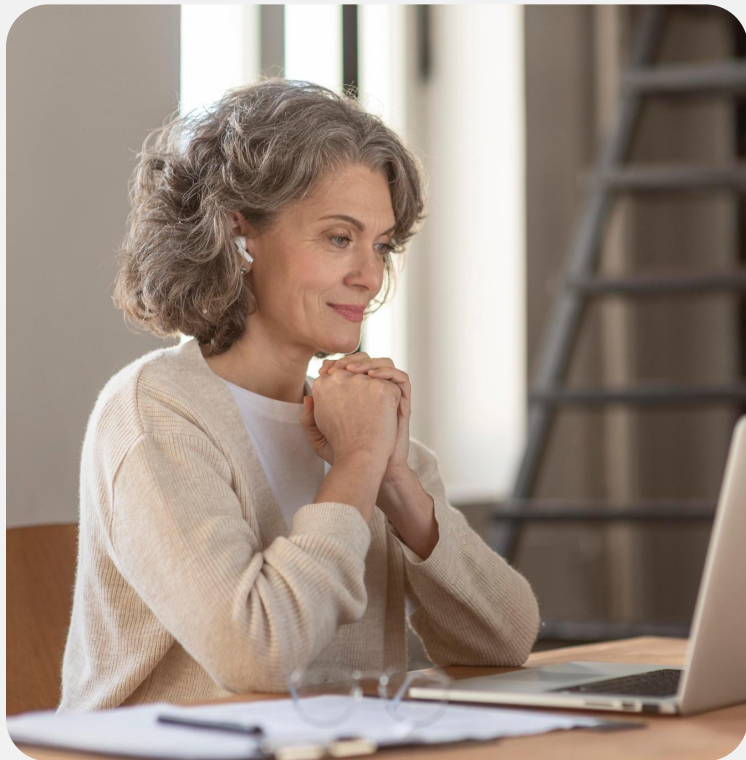


Ответьте на вопрос

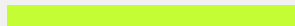
Можно ли аннотировать аргумент, принимающий несколько типов?

Ответ: Да, например можно использовать | начиная с Python 3.10:

```
def example(value: int | str) -> None:
    print(value)
```



Ответьте на вопрос



Можно ли указать `dict[str]` в аннотации?



Ответьте на вопрос

Можно ли указать `dict[str]` в аннотации?

Ответ: Нет, потому что словари всегда содержат пары "ключ: значение", поэтому **нужно указывать оба типа**:

```
def get_data() -> dict[str, int]:
    return {"Alice": 1}
```



Ответьте на вопрос

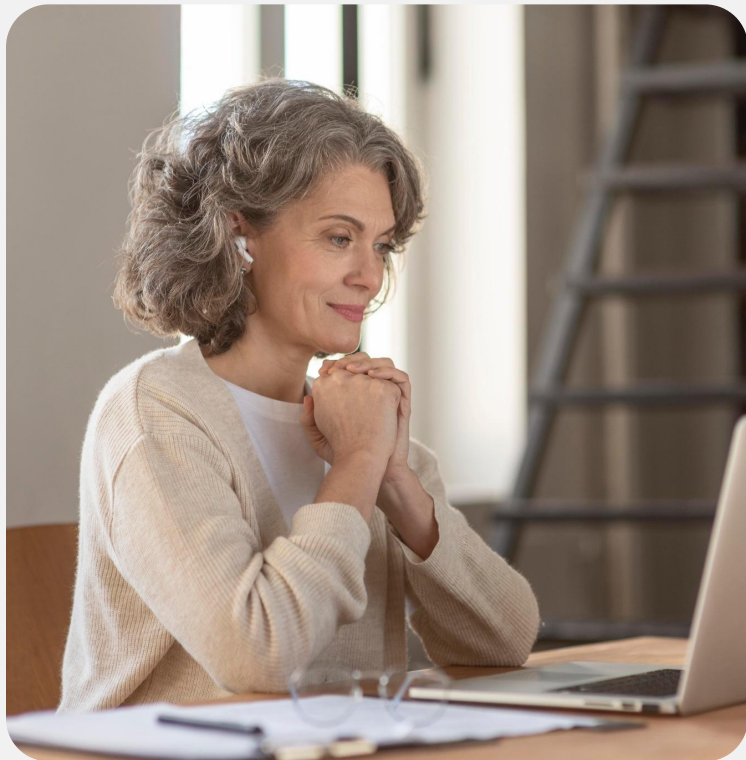
Ответ: `Optional[int]` — это `Union[int, None]`, то есть переменная может быть **целым числом или** `None`



Ответьте на вопрос

Что делает `Optional[int]`?

Ответ: `Optional[int]` — это `Union[int, None]`, то есть переменная может быть **целым числом или** `None`



Ответьте на вопрос

Как передаются аргументы в функцию — по ссылке или по значению?



Ответьте на вопрос

Как передаются аргументы в функцию — по ссылке или по значению?

Ответ: В Python **все аргументы передаются по ссылке**, но неизменяемые объекты (строки, числа, кортежи) создают **новый объект** при изменении.



Ответьте на вопрос

Ответ: Нет, так как **кортеж неизменяемый**. Единственный способ — создать **новый кортеж** и вернуть его.



Ответьте на вопрос

Можно ли изменить кортеж внутри функции?

Ответ: Нет, так как **кортеж неизменяемый**. Единственный способ — создать **новый кортеж** и вернуть его.



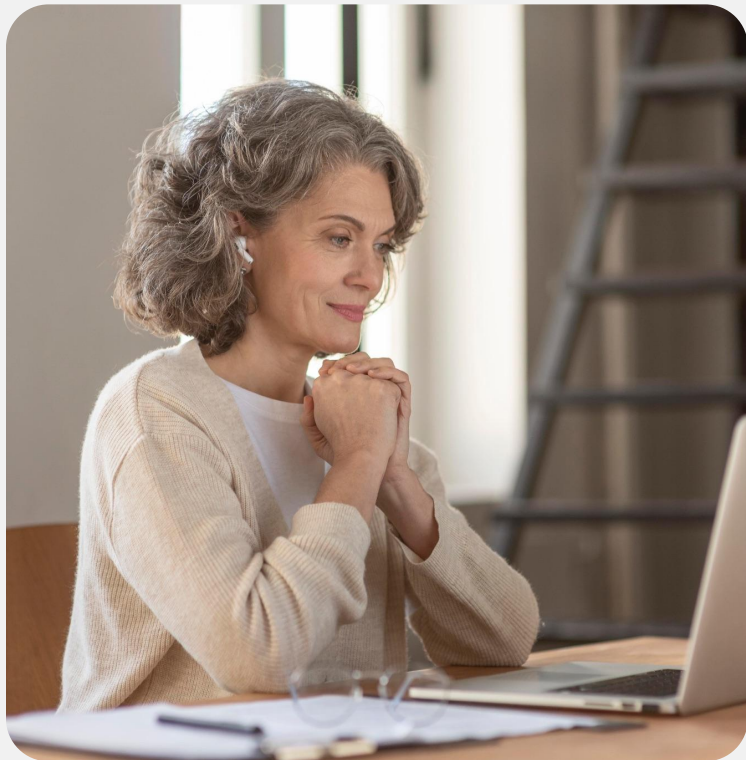
Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_dict(d: dict) -> None:
    d["b"] = 30
```

```
data = {"a": 10, "b": 20}
modify_dict(data)
print(data)
```

- a. {'a': 10, 'b': 20}
- b. {'a': 10, 'b': 20, 'b': 30}
- c. {'a': 10, 'b': 30}



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_dict(d: dict) -> None:
    d["b"] = 30
```

```
data = {"a": 10, "b": 20}
modify_dict(data)
print(data)
```

- a. {'a': 10, 'b': 20}
- b. {'a': 10, 'b': 20, 'b': 30}
- c. {'a': 10, 'b': 30}



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_tuple(tpl: tuple) -> None:
    tpl[0] = 10
```

```
my_tuple = (1, 2, 3)
modify_tuple(my_tuple)
print(my_tuple)
```

- a. (10, 2, 3)
- b. (1, 2, 3)
- c. (10, 1, 2, 3)
- d. Ошибка



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def modify_tuple(tpl: tuple) -> None:
    tpl[0] = 10
```

```
my_tuple = (1, 2, 3)
modify_tuple(my_tuple)
print(my_tuple)
```

- a. (10, 2, 3)
- b. (1, 2, 3)
- c. (10, 1, 2, 3)
- d. Ошибка



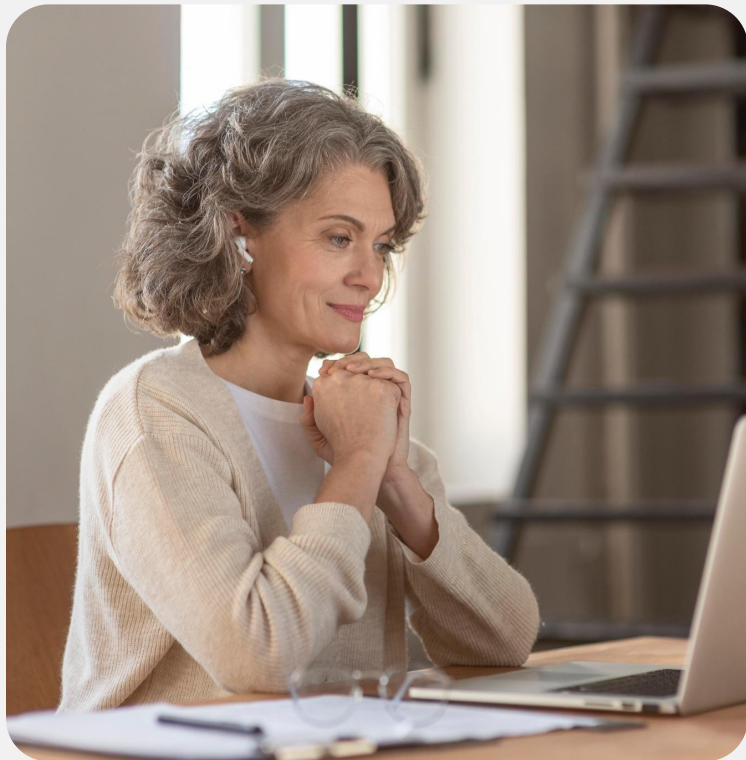
Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def countdown(n: int) -> None:
    if n == 0:
        print("Go!")
        return
    print(n, end=" ")
    countdown(n - 1)
```

- a. 3 2 1 Go!
- b. Go! 3 2 1
- c. 1 2 3 Go!
- d. Go! 1 2 3

countdown(3)



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def countdown(n: int) -> None:
    if n == 0:
        print("Go!")
        return
    print(n, end=" ")
    countdown(n - 1)
```

countdown(3)

- a. 3 2 1 Go!
- b. Go! 3 2 1
- c. 1 2 3 Go!
- d. Go! 1 2 3



Выберите верный вариант ответа

В каком случае использование рекурсии не рекомендуется?

- a. Когда нужно выполнить сложные математические вычисления
- b. Когда количество вызовов функции может быть большим
- c. Когда функция выполняет одно и то же действие
- d. Когда нужно обработать вложенные структуры данных



Выберите верный вариант ответа

В каком случае использование рекурсии не рекомендуется?

- a. Когда нужно выполнить сложные математические вычисления
- b. Когда количество вызовов функции может быть большим**
- c. Когда функция выполняет одно и то же действие
- d. Когда нужно обработать вложенные структуры данных



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def mystery(n: int) -> int:
    if n == 1:
        return 1
    return n + mystery(n - 1)
```

```
print(mystery(5))
```

- a. 15
- b. 10
- c. 5
- d. Ошибка



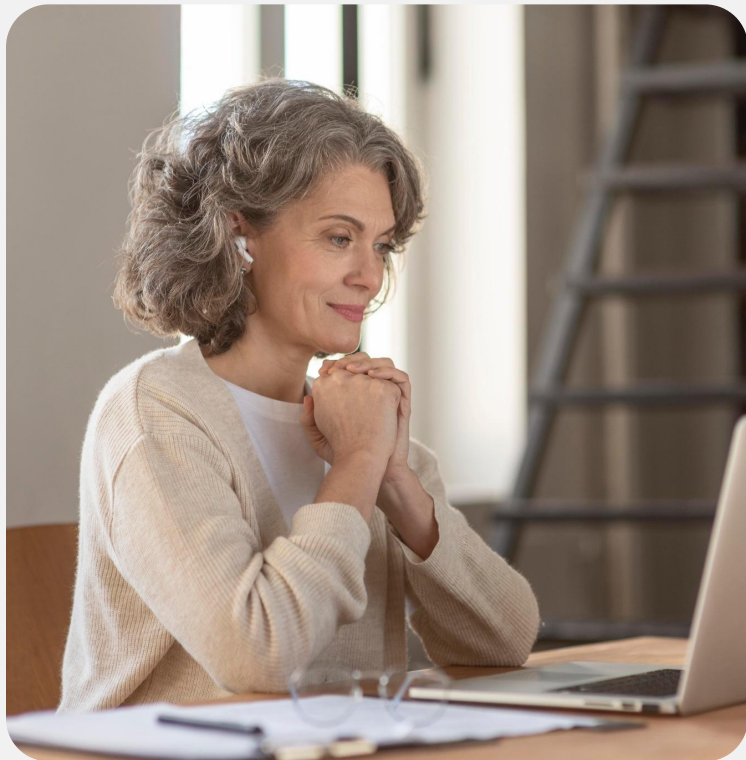
Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def mystery(n: int) -> int:
    if n == 1:
        return 1
    return n + mystery(n - 1)
```

```
print(mystery(5))
```

- a. 15
- b. 10
- c. 5
- d. Ошибка



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def func(n: int) -> None:
    if n > 3:
        return
    print(n, end=" ")
    func(n + 1)
    print(n, end=" ")
```

- a. 1 2 3 3 2 1
- b. 1 2 3
- c. 3 2 1 1 2 3
- d. 3 2 1

func(1)



Выберите верный вариант ответа

Какой результат будет выведен при выполнении следующего кода?

```
def func(n: int) -> None:
    if n > 3:
        return
    print(n, end=" ")
    func(n + 1)
    print(n, end=" ")
```

- a. 1 2 3 3 2 1
- b. 1 2 3
- c. 3 2 1 1 2 3
- d. 3 2 1

func(1)



ВОПРОСЫ





РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

Объединение данных в строку

Напишите функцию, которая принимает список любых данных (строки, числа, списки, словари) и возвращает их строковое представление, объединённое через " | ". Добавьте документацию и аннотации типов для всех параметров и возвращаемого значения.

Данные:

```
data = [42, "hello", [1, 2, 3], {"a": 1, "b": 2}]
```

Пример вывода:

```
42 | hello | [1, 2, 3] | {'a': 1, 'b': 2}
```

Домашнее задание

Сумма вложенных чисел

Напишите функцию, которая принимает список словарей, где каждый словарь содержит имя пользователя и список баллов. Функция должна вернуть сумму всех чисел. Добавьте документацию и аннотации типов для всех параметров и возвращаемого значения.

Данные:

```
data = [  
    {"name": "Alice", "scores": [10, 20, 30]},  
    {"name": "Bob", "scores": [5, 15, 25]},  
    {"name": "Charlie", "scores": [7, 17, 27]}  
]
```

Пример вывода:

Итоговый балл: 156



ВОПРОСЫ



Домашнее задание

Сумма цифр числа

Напишите рекурсивную функцию, которая находит сумму всех цифр числа

Данные:

num = 43197

Пример вывода:

24

Домашнее задание

Сумма вложенных чисел

Напишите рекурсивную функцию, которая суммирует все числа во вложенных списках.

Данные:

```
nested_numbers = [1, [2, 3], [4, [5, 6]], 7]
```

Пример вывода:

28



ВОПРОСЫ



Заключение

